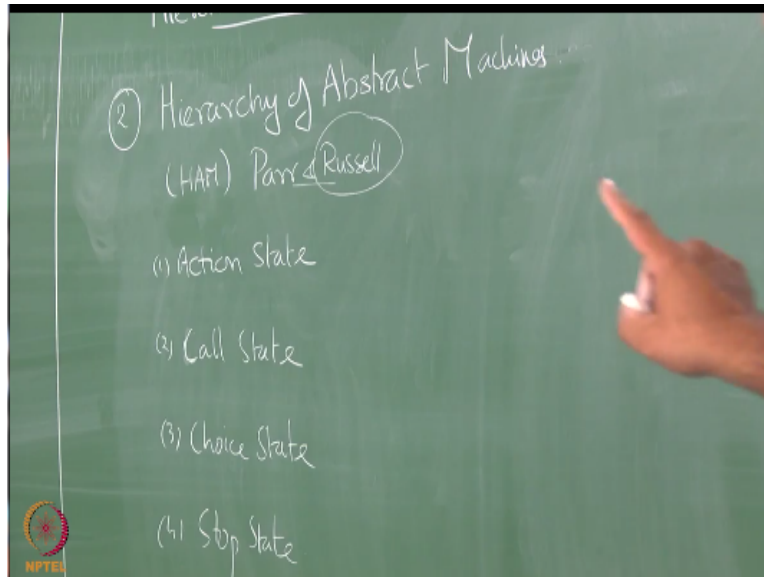


**NPTEL**  
**NPTEL ONLINE COURSE**  
**REINFORCEMENT LEARNING**  
**HAM**

**Prof. Balaraman Ravindran**  
**Department of Computer Science and Engineering**  
**Indian Institute of Technology Madras**

(Refer Slide Time: 0:15)



So the second framework which is actually a very interesting thing unfortunately is not adopted very widely is called the hierarchy of abstract machines or HAM okay in fact HAMs have become more like a curiosity in the RL community that's what I, at least I do not see anybody write papers about HAMs or anything, you talk to people who are working in RL literature right they would have read up on HAMs at some point but never given it very serious consideration after that but so Andy Barto thinks that the HAM framework is the most elegant and most complete of all the three major hierarchical RL frameworks that are out there unfortunately people just have not built a significant structure over it except for people who work with Stuart Russell right.

So it was proposed by Ronn Parr right so yeah the options frame work was proposed by Doina Precup Rich Sutton and Satinder Singh so the original options paper was Sutton Precup Singh is the citation for that and it was largely largely Doina's work yeah I mean when I start talking about options I become a little nostalgic because that is right about the time I was starting my PhD as well so I have seen like hundred versions of options before it finally came into this form right so they talked about alpha models beta models and then beta time models and they have a whole bunch of different versions of options before it kind of crystallized into this right.

And this whole connection of all of these frameworks options framework HAM and max Q and all of these frameworks the connections to SMDP Q learning were actually originally observed by Andy Barto so somebody was giving a presentation in one of the meetings and then Andy just said oh that is SMDPs right and then people went and clamored around and then they actually figured oh yeah okay he's right I mean we are just in some sense many of the results that they were showing at that point were reinventing the wheel that people have done with SMDPs so anyway so going back to HAMs I believe the ham framework was the first hierarchical RL framework that came out I think in 96 or something 97 I think but the options framework became more popular later and mostly the work all the follow-up work have been done in Stuart Russell's lab and even Ron Parr did not continue to work on HAMs after that but it is a really cool thing right and it is also very complex.

So what do I do with HAMs is that I am going to define so what I call are simple machines that encode the policies that I am trying to follow right machines finite state machines they are like finite state machines I am going to define a hierarchy of finite state machines okay that is what we call the hierarchy the hierarchy and the machines part is all clear the abstract part will become clear in a minute okay so I am going to define hierarchy of finite state machines right so the finite state machines will have the following types of states four types of states okay.

So the first state will be what is called an action state right the second is called a call state third is called a choice fourth is stop okay so what does the action state do it takes an input emits an action okay what does the call state do takes an input, calls another machine right what does the

choice state do that is a confusing thing right what is that exactly the choice state take well choice it has to be a decision right so what do you do with that what do you mean decision think of it in terms of finite state machines what could it be how will you encode decision making in a finite state machine hmm close you may give a non-deterministic transition right.

So essentially a choice state makes a non-deterministic transition to two or more states in the same machine okay so a call state calls a another machine but a choice state transitions to one or more states non-deterministically in the same machine and what does the stop state do you think stops and then returns control back to the machine that called it right so because machine would be called by a call state okay and there is also this starting machine right which is where you start your policy makes sense what is the input that you are going to see for these machines states right.

The underlying states of the MDP so the inputs to the machines are the underlying states of the MDP right the outputs on these machines are come on what do action states do yeah that is the output primitive actions primitive actions it is going to take as input primitive states as input it is going to output primitive actions right.

So the only output happens in the action state so the call state does not output anything what does the call state do it just transfers control to another machine so you keep going to one machine to another machine until you hit an action state, when you hit an action state you emit an action and that is when you actually move in the real world okay makes sense so this is how things will work so I have this hierarchy right I have this machine's well how does the hierarchy get established here the calls right.

So the calls will give me a hierarchy I will have to make sure it is a proper hierarchy I should not have loops right if I have loops then I can just keep calling around in the loop and they can get into infinite depth into my policy so make sure it is a proper call graph right so it should be a dag right so make sure my machines are connected in a dag right and then I start in the original machine I look at what the initial state is right that is the top-level machine let us call it some M not so M not will take a state right.

And then the start state in M not will now make a transition it could be the start state could be a choice state it could be an action say it could be a call state whatever it sees the input and based on the input it makes a call right so it could start off 1 machine or the other and then that machine now makes another call again finally in some level in some machine I am going to get a primitive action if you think this is very complex just think about how we talked about the getting out of the going to a central station example right.

There is one big machine that is going to central station okay it takes as input okay you are in the ICSR building right then it is going to say okay let me call the get out of insti machine so I will go to the gate out of insti machine get out of insti machine is going to look at the state hmm you are in the ICSR building okay go to GC machine then the go to GC machine will look at the state hmm you are in the ICSR building okay and then it will call get out of ICSR building and that will go down it will call the get out of the class room machine right.

And the get out of the classroom machine will say oh finally ok move right that will be the first primitive action you will do right so all of this processing happens and then you move right and then what happens I am still in the get out of class room machine because that is not reached a stop state right I keep moving until I reach the goal I just do not want to exit and walk out to demonstrate the point but you see what I mean right as soon as I reach the door I will actually get into a stop state in the get out of classroom machine.

It will pop up and go to the get to yeah memory is so low is it, what machine called it, get out of the ICSR building right so that is the machine that calls this it will go to the get out of ICSR building now what it will do is yeah okay you are no longer in the classroom right it will see that then it will say okay go down the stairs machine right and the go down the stairs machine will then allow me to take my actions one after the other so it will give me a series of primitive actions I will walk down the stairs and then it will reach a stop state.

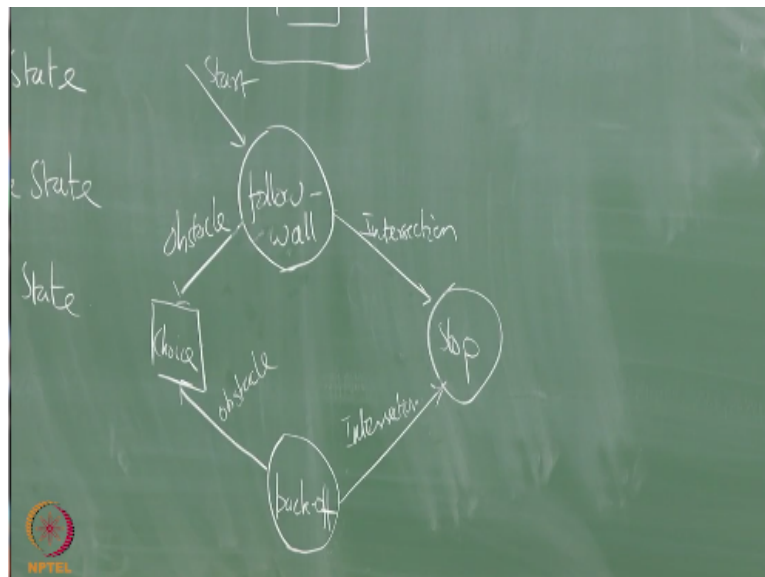
And it will pop back up to the gate out of building machine and then it will finally say okay walk out of the door machine then it will go back and call that machine it will go back up and now I

will make a transition in the get out of go to the GC it will go backup I am already out of the building so it will go back one more level up and it will say great now I can go to the GC right it will call the go to GC machine and then give an analogy.. there you go.. is the lift working if the lift is working then.

I go to the I will go to an action that says take the lift if the lift is not working I will go to the action that says step down that is a choice state no I am just I am defining go to the lift as a primitive action and it could be a choice state now it could be a call state the choice state can make a transition to a call state I did not see it has to make a transition to a primitive state so the choice state I will actually draw a simple so this is easier to connect to right whatever I described you can understand how HAMs are going to be used and stuff but there is also an example from the paper so how many of you looked at the recent advances in hierarchical RL paper.

That has already been on on moodle since the last class read the first four sections you do not even have to read the fifth section okay yeah

(Refer Slide Time: 13:46)



So going back let us let us take another example that they have here so the problem was to do some kind of a very complex so they had a very complex grid world task okay each of these corridors will be divided so there will be like 10 grid cells here right this is a pretty large grid well right so some 300 x 400 grids or something like that and all along the corridor okay there were small obstacles right in a specific shape there is like this shape like this they put obstacles all around the corridor.

So whenever you are going around right so you may hit an obstacle and you are supposed to find your way around the obstacle you are trying to learn your way around the obstacle so what you can either say that you are going to follow the wall you know so if I hit the obstacle here I can essentially follow the wall and I will get out right or I can back up go back the way I came turn in arbitrary direction and move out to see if I evade the obstacle ok these are the two ways in which you can get around the obstacle.

And so they wanted to come up with a machine that learns how to do this right so what they did was they defined it as follows so they have a they have a follow wall yeah sure 300 cross 400 a lot of it is repeated right and once you know how to avoid one of those that bracket shaped obstacles then why do you have to relearn everything again and again right this is the whole idea going to hierarchies that you do not want to keep learning the same thing again and again right so I am going to follow the wall right start somewhere I just keep following the wall then what I do right so if I hit an intersection so this is a call state okay.

So I am calling the machine for following the wall so there might be some things that needs I need to do because if you assume it is actually a robot I might need to position myself a distance away from the wall and then keep moving and things like that so when the follow wall command the machine returns I will be in some state right so I could be either in an intersection I am assuming that as long as I can validly follow the wall this machine will run right when this when it comes back to this machine right when the follow wall machine terminates and comes back to this machine I will either be at an intersection or I will be at a obstacle.

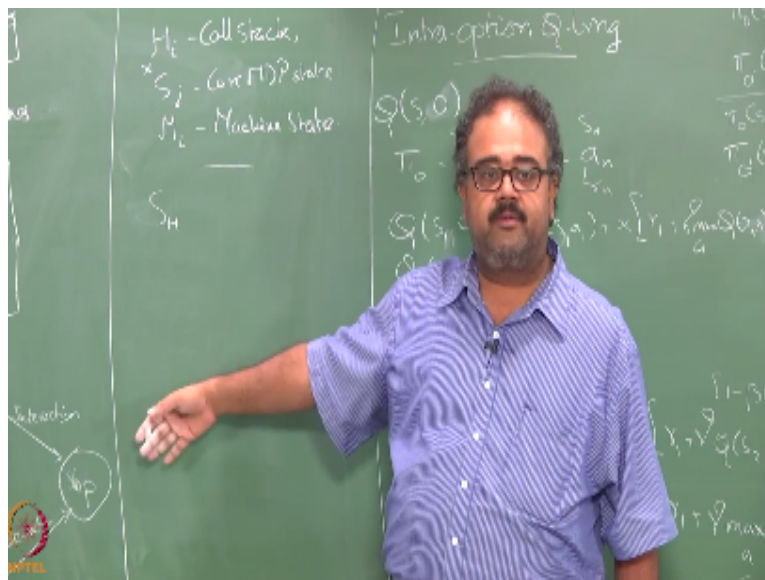
At an intersection I stop basically I go back up because when they come to an intersection right now I have to make a choice of whether I have to go this way that way or this way right so that choice will be determined by a different machine okay so my job here was to walk from this point to this point that is the goal of this machine you can think of this as navigate a corridor ok this machine is essentially navigating corridors right so if I start it off here it will go all the way here it will stop here and it will tell me okay I am done and if you start it off here may be it will go here or here one of those two things you give it a proper direction you say go in this direction it will go all the way here it will stop and then it will come back okay.

So that is the goal of this machine it will navigate a corridor okay so it will stop follow wall will stop if it reaches an intersection because there is no wall here so it will stop ok it will say okay I have reached the intersection and then if I reach the intersection I am happy I will just stop and we will give it back if I reached an obstacle then what do I do I have a choice okay so what can I do I can two choices I have I can pick a different wall and follow it right or I can back off these are the two things I can do.

So what is back off back off is another machine like I was telling you it moves me back in the direction I came okay so this is remember this is not a real machine okay and just this is a sample thing so the choice state I can either say follow wall okay or I can say back off or I can even say stop if you want I mean I do not care right so the choice state allows me to go to any one of these three things and what do I do with backoff so it is another machine I call it when it returns I either reached another obstacle or I have reached an intersection okay.

If I reach an intersection I stop basically it could very well be that I went all the way back so that I went back to where I started right or I reached another obstacle so I came back and try to do another choice stake pick a different wall and follow or whatever right so sorry what is that choice I make it work it just says choice this is a non-deterministic transition so in this case it can go to any one of the other three states in the machine it could go to follow wall it could go to back off or it could go to stop or you can limit it you can say no you cannot stop you can go to only one of the other two options.

(Refer Slide Time: 20:08)



That I can say that okay you can either go here that is your non deterministic transition so from choice you can either go to follow wall or you can go to stop do not think of it as a realistic way of solving this problem please okay right just a simple illustrative machine it might not be the actual machine okay great any questions on this let me at least try and finish this today and then I can start max Q tomorrow so give me a few more minutes follow another machine just like a



hierarchy that is what these are call states they are call states action will happen somewhere down there right.

So follow wall might say okay rotate my motor no no no follow wall is another machine like this okay so there will be a follow wall machine okay that will come somewhere like this right so maybe not a choice here I do not know we do not have a symbol for action States it will be some action it will say move right there might be actions for moving right moving left or whatever so you come here and then you decide whether to move right move left and so on so forth.

So it will be another machine with action states so this machine has no action states all fine fine everybody fine on board what is that the call states are follow wall and backoff okay so in this we have call states choice states stop states no action states the action states will come further down somewhere which will be things actions like move left move right or turn motor so much depending on whatever it is your primitive action okay so that will come in some other machine we are not bringing about that here is my question to you where is RL here what do you learn here where is the SMDP here I am talking about a different hierarchical framework it is no longer options here you should forget options while I am talking about hams there are primitive actions but where are my where is my SMDP here.

So what do what do you think I should be learning which machine to call or essentially what do I do in the choice state right choice state need not be machine right it could be other action states also it could be I could come to a choice state and then say okay if I am here then go left if I am here go right I mean I can look at the actual the what they call it the core MDP state the basic the base MDP state basically that is the input I am getting right I can look at the input in the choice state I can say go left go right I mean it could be primitive actions also I could be transitioning to action states.

So I will have multiple choices that I could do in the choice state I really have to learn what is this choice I should make right but where is the SMDP that I am going to learn on where is the SMDP give me a specification of the SMDP so what are the states of the SMDP so if you remember in the options case the states of the SMDP were are the same as the original states

right and the actions of the SMDP were essentially the actions plus options I told you right you might have 4 actions and 6 options totally the SMDP will have ten actions right so likewise where is the SMDP in this case what is the state space.

The call states and the choice states no the underlying MDP is we can throw it away is it that does not form part of the SMDP so I need to have my base MDP states in my SMDP because I have to learn a policy that is a function of where I am in the world but I cannot ignore the base MDP state or the core MDP state I need the core MDP state but what action I pick also depends on what machine I am in right it not only depends on what machine I am in it also depends on who called it right once you are outside the building here depending on whether you are going to the main gate or velocity gate depending on which machine higher up called down called the get out of building machine I would have to take a different action right it does not just depend on whether I am in the get out of building machine it also depends on which machine called me.

Right in fact there is a stack I have to keep a stack I have to keep a call stack which machine called which machine which machine called which machine so that call stack has to be part of the state space because what decision I make depends on the entire call stack so now what are the things that need to be part of the state space the state of the base MDP and the call stack right great so now what are my actions for my SMDP what is my action space is it the actions of the core MDP what is that you have states and have the call stack no no first answer my second question that is the reason I did not answer his question I cannot answer this question until you answer my question what are the actions here I will tell you primitive actions are not the actions that I am going to look at in this SMDP.

I will ignore the primitive actions in the SMDP because the primitive actions are all already predetermined if I land in an action state I will emit a primitive action right I have no choice over that I am giving you hints so what should be the actions transitions possible out of the choice state therefore two of those actions will be follow wall and backoff there will be two of those actions that will go into my SMDP right once a call follow wall it is like an option I have called now it will keep running until it reaches a intersection maybe there might be other choice states

within the follow wall option in which case it will come back to me and I will have to make a decision what to do that okay.

So essentially the SMDP consists of I do not know I am going to call this as  $H_i$  as the call stack right it is the history of the things I have called it is essentially the call stack right and  $S_i$  which is the core MDP state right these are the this cross this right so that will be my action space there will be the call stack times the current state that will be my state space right and my action space would be all the outcomes of the choice state  $C_i$  right so all the possible outcomes of the choice state  $C_i$  will be the actions I can take right and of course there will be some limitations I can take a particular action only if I am in a choice state corresponding to that machine right.

So I need to know which choice state I am in so I not only need the call stack what else do I need I need to know what state machine state I am in as well no no no the choice is the call state could be an action state provided it is something which I can reach to from a choice state if I have a machine that is completely specified without any choice state none of the states in the machine will appear in my actions right I could define a machine without a choice state I mean I could know the policy completely beforehand I do not have any learning to do for a particular policy for example we have been talking about options right.

So if I am trying to encode the policy of an option it will be a machine without a choice state suppose I have defined an option get out of this room right the machine for corresponding to that will look like okay sense the world right so okay there is a if you are in this state then back off right until you reach here and then turn right and then walk out right so it will be essentially there will be no choice states here this entire machine will have no choice states right it is possible and none of these things will actually appear in the none of the states.

That I have in this machine will appear in the actions for the SMDP right no no the choice could depend on the call stack right depending on where you are you can choose to go left or right depending on their call stack could could that is what it gives you the freedom for defining it that way. I am not talking about learning recursively optimal policies here I could learn hierarchically optimal policies when I am using HAMS it is actually pretty powerful framework allows you to

do all kinds of things in the options thing that question does not even arise because I have frozen my hierarchy right.

We are talking about this in the last class also it kind of becomes degenerate right the recursively optimal is equal to hierarchically optimal in the options case because I cannot change the policies we have to assume whatever policy is given to us is optimal but here I can actually if I act if I start conditioning it on the call stack and they can potentially think of learning hierarchical optimal policies okay is it fine so I need the machine state also.

I need to know which choice state I am in there could be multiple choice states in a machine right I need to know which choice state I am in also and so I take all of this together and then my actions are essentially other machine states what to call it right so  $S_m$  are machine states so actions are this machine states it could be one of these states that I have to go to right so this is essentially my SMDP and the transition functions are defined by okay if I choose to go from here the action I take is follow wall then that machine will call it will keep running till the end of that machine's execution and then it will come back here okay.

So that is essentially the transitions yeah machine states are these finite state machine states one of these machine states which state I am here right when I actually come here which is the state I am in when I make the call no that is the machine I am in this will be the machine state I am in so which are which is the path through the tree would be there the dag would be that and this will be where in this machine I am okay so I mean I might be kind of fudging the notation a little bit you can read the paper and find out what the exact notation is but so that is I am just trying to give you the intuition not trying to knock down lock down everything.

There was one other thing which I wanted to tell you and how would now that you know what are the states right and what are the actions right you can learn policies using SMDP Q learning right so what is the weird thing about this ahh this is another thing I wanted to mention so whenever I make a call action right or a choice action right whenever I go to a call or choice state there is no transition in my core MDP state right only the machine state and the call stack would change.

So the only time the core MDP state changes is if I visit the action state anywhere in the in any of the machines I do not care right whenever I am in the process of executing the policy if I reach an action state then right the core MDP state will change otherwise that component will stay the same right and only these components will change okay so this is essentially the idea right and so the thing about this is so what will be the equivalent of an option in this case.

So the option encapsulates an already learnt policy right so what is the equivalent of an already learnt policy here hmm without, without any choice states what without any choice states any sequence of action states you visit without any choice state in between right I mean those would constitute your policy or an execution of the policy because when you are running this machine if there is a sequence of states that you visit sequence of action states you visit without a choice state in between right that will constitute a learnt policy makes sense because that is that is already prepackaged I have no choice there that is like an option once a pick the option I just have to take the action suggested by the option so like that.

So that is the thing so as you can see this is a little little more complex than the options framework right so people are not very did not take to it too kindly but if you think about it is a very very powerful framework it allows you to work in partially observable states right so when you do not need to know the full state spaces because one of the nice things is you can use this kind of finite state machines to define more complex policies right you can define non Markov policies easily you can have finite state machines with memory right you can define non Markov policies so the representation itself is very powerful but unfortunately this gets very complex so people are not very fond of this I will stop here I will continue with the max Q thing and also more discussions on hierarchy specific abstractions and other things in the next class.

**IIT Madras Production**

**Funded by**

**Department of Higher Education**

**Ministry of Human Resource Development**

**Government of India**

[www.nptel.ac.in](http://www.nptel.ac.in)

**Copyrights Reserved**